

# **INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)**

## **Lab 1 - HTTP Analysis Using Wireshark: Text Traffic**

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>Lab Title:</b>    | <b>HTTP Analysis Using Wireshark</b>                 |
| <b>Student Name:</b> | <b>Yusuf Oluwaseun Dada</b>                          |
| <b>Student ID:</b>   | <b>2025/FWSD/11339</b>                               |
| <b>Course Code:</b>  | <b>SBT-DF203</b>                                     |
| <b>Course Name:</b>  | <b>Basic Networking Skills for Digital Forensics</b> |

**Instructor: Mr. Aminu Idris, AMCPN**

**Date: 08/09/2026**

## Required Tools and Environment

| Item                 | Recommended Tool       | Purpose                                            |
|----------------------|------------------------|----------------------------------------------------|
| Operating system     | Kali Linux             | Run Apache, curl, tshark and Wireshark             |
| Web server           | Apache2                | Serve the training webpage locally                 |
| Capture tools        | Tshark and wireshark   | Capture and inspect network packets                |
| Client               | curl and a web browser | Generate controlled HTTP requests                  |
| Hashing              | sha256sum              | Verify capture integrity                           |
| Optional second host | Linux VM               | Produce Ethernet frames with visible MAC addresses |

## Step 1 - Lab Folder Structure and Environment Setup

I create the folder structure **before** downloading or generating any evidence. Store original captures under evidence/ and analysis copies under working/ not to analyze the original directly.

```
(kali@kali)~[/Desktop]
$ mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
/home/kali/SBT-DF203-Lab1

(kali@kali)~[/SBT-DF203-Lab1]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
find . -maxdepth 1 -type d -print
sudo apt update
sudo apt install -y apache2 curl wireshark tshark
```

```
(kali@kali)~[/Desktop]
$ mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
/home/kali/SBT-DF203-Lab1

(kali@kali)~[/SBT-DF203-Lab1]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali)~[/SBT-DF203-Lab1]
$ sudo systemctl enable --now apache2
[sudo] password for kali:
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apache2.service'.

(kali@kali)~[/SBT-DF203-Lab1]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>\n' | sudo tee /var/www/html/basic.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>
```

```
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>\n' | sudo tee /var/www/html/basic.html
```

```
(kali@kali) - [~/SBT-DF203-Lab1]
$ curl -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 36726
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Mon, 07 Sep 2026 21:43:47 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Mon, 07 Sep 2026 21:40:12 GMT
< ETag: "89-65aeb787ead21"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact
```

curl -v http://127.0.0.1/basic.html

## Mini Evidence and Chain-of-Custody Worksheet

| Field                       | Entry                                                            |
|-----------------------------|------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab1                                                   |
| Trainee name                | Yusuf Oluwaseun Dada                                             |
| Date and time started       | 06/09/2026 9:00pm                                                |
| Evidence file name(s)       | basic.pcapng                                                     |
| Source or generation method | Https Analysis                                                   |
| Original SHA-256            | dcbc5dc0ac0a794d3ff1a4813be2419a58e9db47fa150e0932baf2bce8e35711 |
| Working-copy SHA-256        | dcbc5dc0ac0a794d3ff1a4813be2419a58e9db47fa150e0932baf2bce8e35711 |

## Part A - Verify the Web Service and Generate Text HTTP Traffic

1. Replace the name and registration number placeholders in basic.html with your own details.

```
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>\n' | sudo tee /var/www/html/basic.html
```

2. Open <http://127.0.0.1/basic.html> and confirm that the page displays correctly



3. Run curl with verbose output and save the request/response headers.
4. Record the local service state and listening port.

```
sudo systemctl status apache2 --no-pager
```

```
sudo ss -lntp | grep ':80'
```

```
curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
```

```
(kali@kali)~[~/SBT-DF203-Lab1]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-08 08:51:43 EDT; 17min ago
     Invocation: 5e8eb40d90d8492a8342e1c75a87340d
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 907 (apache2)
      Status: "Total requests: 2; Idle/Busy workers 100/0; Requests/sec: 0.00188; Bytes served/sec: 1 B/sec"
     Tasks: 7 (limit: 2099)
  Memory: 18.6M (peak: 34.3M, swap: 5.5M, swap peak: 5.5M)
       CPU: 1.736s
    CGroup: /system.slice/apache2.service
            └─ 907 /usr/sbin/apache2 -k start -DFOREGROUND
              5016 /usr/sbin/apache2 -k start -DFOREGROUND
              5017 /usr/sbin/apache2 -k start -DFOREGROUND
              5018 /usr/sbin/apache2 -k start -DFOREGROUND
              5019 /usr/sbin/apache2 -k start -DFOREGROUND
              5020 /usr/sbin/apache2 -k start -DFOREGROUND
              6851 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 08:51:28 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 08:51:42 kali apachectl[907]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0...his message
Sep 08 08:51:43 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Sep 08 08:59:07 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Sep 08 08:59:19 kali apachectl[4895]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127...his message
Sep 08 08:59:19 kali systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo ss -lntp | grep ':80'
curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
LISTEN 0      511          *:*        users:((("apache2",pid=6851,fd=4),("apache2",pid=5020,fd=4),("apache2",pid=5019,fd=4),("apa
che2",pid=5018,fd=4),("apache2",pid=5017,fd=4),("apache2",pid=5016,fd=4),("apache2",pid=907,fd=4))
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 34882
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
0         0     0    0      0     0      0      0      0      0     0      0      0      0      0      0      0      0      0      0      0      0
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 13:22:00 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Mon, 07 Sep 2026 21:40:12 GMT
< ETag: "89-65aeb767ead21"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [137 bytes data]
100  137 100  137    0     0  531    0      0      0      0      0      0      0      0      0      0      0      0      0      0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>

```

Activate Windows  
Go to Settings to activate Wi

## Part B - Capture the Session

I used tshark. The loopback interface is normally lo. I Start the capture **before** running curl, and stop it **after** the page is received.

```

(kali@kali)-[~/Desktop]
$ curl --no-keepalive -v http://127.0.0.1/basic.html
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 36220
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 13:37:41 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Mon, 07 Sep 2026 21:40:12 GMT
< ETag: "89-65aeb767ead21"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

```

# Terminal 1

sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng

```

(kali@kali)-[~/Desktop]
$ curl --no-keepalive -v http://127.0.0.1/basic.html
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 56828
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 14:06:59 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Mon, 07 Sep 2026 21:40:12 GMT
< ETag: "89-65aeb767ead21"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Yusuf Oluwaseun Dada</p><p>Reg No:2025/FWSD/11339</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

```



## # Terminal 2

`curl --no-keepalive -v http://127.0.0.1/basic.html`

```
└─# tshark -r basic.pcapng
Running as user "root" and group "root". This could be dangerous.
 1 0.000000000    127.0.0.1 → 127.0.0.1    TCP 74 56828 → 80 [SYN] Seq
=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3814739275 TSecr=0 WS=256
 2 0.000024722    127.0.0.1 → 127.0.0.1    TCP 74 80 → 56828 [SYN, ACK
] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=224031055 TSecr=3
814739275 WS=256
 3 0.000042863    127.0.0.1 → 127.0.0.1    TCP 66 56828 → 80 [ACK] Seq
=1 Ack=1 Win=65536 Len=0 TSval=3814739275 TSecr=224031055
 4 0.010061132    127.0.0.1 → 127.0.0.1    HTTP 149 GET /basic.html HT
TP/1.1
 5 0.010149008    127.0.0.1 → 127.0.0.1    TCP 66 80 → 56828 [ACK] Seq
=1 Ack=84 Win=65536 Len=0 TSval=224031065 TSecr=3814739285
 6 0.167049859    127.0.0.1 → 127.0.0.1    HTTP 454 HTTP/1.1 200 OK (
text/html)
 7 0.167094553    127.0.0.1 → 127.0.0.1    TCP 66 56828 → 80 [ACK] Seq
=84 Ack=389 Win=65280 Len=0 TSval=3814739442 TSecr=224031222
 8 0.167478706    127.0.0.1 → 127.0.0.1    TCP 66 56828 → 80 [FIN, ACK
] Seq=84 Ack=389 Win=65536 Len=0 TSval=3814739443 TSecr=224031222
 9 0.167833442    127.0.0.1 → 127.0.0.1    TCP 66 80 → 56828 [FIN, ACK
] Seq=389 Ack=85 Win=65536 Len=0 TSval=224031223 TSecr=3814739443
10 0.167864148    127.0.0.1 → 127.0.0.1    TCP 66 56828 → 80 [ACK] Seq
=85 Ack=390 Win=65536 Len=0 TSval=3814739443 TSecr=224031223
11 60.369562052    127.0.0.1 → 127.0.0.1    TCP 74 42334 → 80 [SYN] Se
q=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=2917298426 TSecr=0 WS=256
12 60.369649833    127.0.0.1 → 127.0.0.1    TCP 74 80 → 42334 [SYN, AC
K] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=677493176 TSecr=
2917298426 WS=256
13 60.369684156    127.0.0.1 → 127.0.0.1    TCP 66 42334 → 80 [ACK] Se
q=1 Ack=1 Win=65536 Len=0 TSval=2917298426 TSecr=677493176
```

Confirmation of the received packet in basic.bcampng file

## Preservation of the working copy

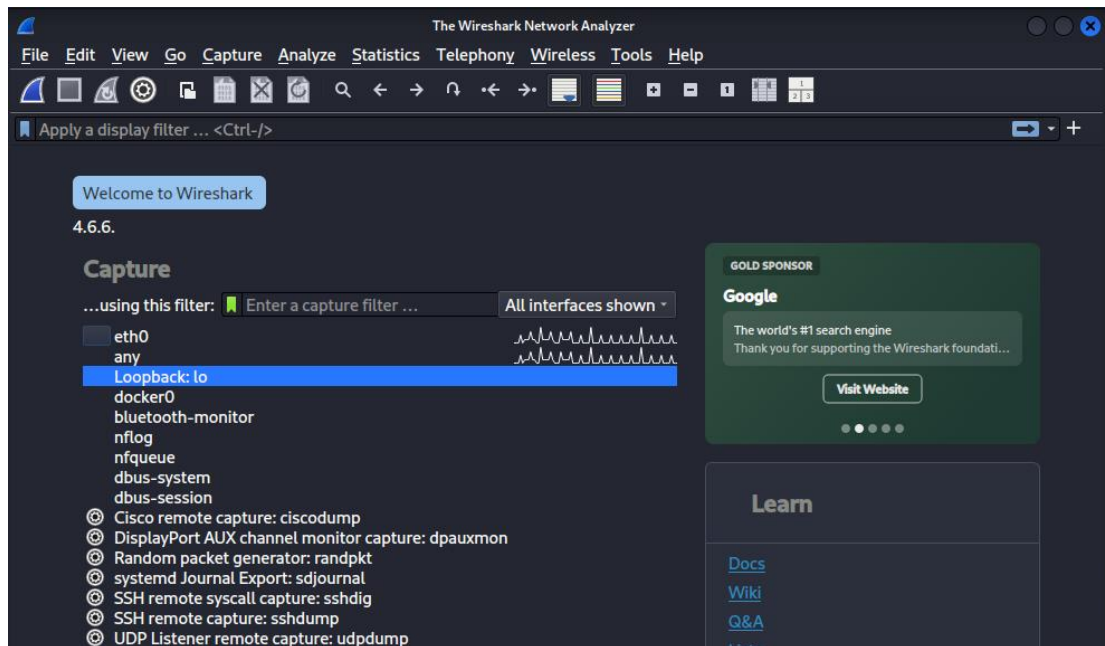
```
(root@kali)-[/home/kali/GBT-DF203-Lab1]
└─# cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng
sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
37ce6a924428ad9e2ef016aa541aa53c64b2445fd2e2be2ff839802538dd975b evidence/basic.pcapng
37ce6a924428ad9e2ef016aa541aa53c64b2445fd2e2be2ff839802538dd975b working/basic_working.pcapng
```

`cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng`  
`sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt`

```
(root@kali)-[/home/kali/GBT-DF203-Lab1]
└─# sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt
dcbcd5dc0ac0a794d3ff1a4813be2419a58e9db47fa150e0932baf2bce8e35711 evidence/basic.pcapng
```

The SHA-256 of the capture

`sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt`



## Wireshark as alternative

### Part C - Analyze the TCP Three-Way Handshake

I Open the working copy and apply the display filter `tcp.stream eq 0`. and identify the first three packets to complete the table below.

| Packet | Flags    | Client Seq | Server Seq/ACK | Timestamp                          | Interpretation                                                |
|--------|----------|------------|----------------|------------------------------------|---------------------------------------------------------------|
| 1      | SYN      | 0          | -              | 2026-09-08T10:06:59.024653496-0400 | Client requests a TCP session.                                |
| 2      | SYN, ACK | -          | 0/ACK 1        | 2026-09-08T10:06:59.024678218-0400 | Server acknowledges and provides its initial sequence number. |
| 3      | ACK      | 1          | ACK 1          | 2026-09-08T10:06:59.024696359-0400 | Client acknowledges; the connection becomes established.      |

```
sudo tshark -r working/basic_working.pcapng -Y "tcp.flags.syn==1||(tcp.flags.ack==1 && tcp.len==0)" \
```

```
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
```

```
-e tcp.flags.str -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
```



```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo tshark -r working/basic_working.pcapng -Y "tcp.flags.syn==1||(tcp.flags.ack==1 && tcp.len==0)" \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags.str -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
1 2026-09-08T10:06:59.024653496-0400 127.0.0.1 56828 127.0.0.1 80 .....S. 0 0
2 2026-09-08T10:06:59.024678218-0400 127.0.0.1 80 127.0.0.1 56828 .....A..S. 0 1
3 2026-09-08T10:06:59.024696359-0400 127.0.0.1 56828 127.0.0.1 80 .....A.... 1 1
5 2026-09-08T10:06:59.034802504-0400 127.0.0.1 80 127.0.0.1 56828 .....A.... 1 84
7 2026-09-08T10:06:59.191748049-0400 127.0.0.1 56828 127.0.0.1 80 .....A.... 84 389
8 2026-09-08T10:06:59.192132202-0400 127.0.0.1 56828 127.0.0.1 80 .....A...F 84 389
9 2026-09-08T10:06:59.192486938-0400 127.0.0.1 80 127.0.0.1 56828 .....A...F 389 85
10 2026-09-08T10:06:59.192517644-0400 127.0.0.1 56828 127.0.0.1 80 .....A.... 85 390
11 2026-09-08T10:07:59.394215548-0400 127.0.0.1 42334 127.0.0.1 80 .....S. 0 0
12 2026-09-08T10:07:59.394303329-0400 127.0.0.1 80 127.0.0.1 42334 .....A..S. 0 1
13 2026-09-08T10:07:59.394337652-0400 127.0.0.1 42334 127.0.0.1 80 .....A.... 1 1
15 2026-09-08T10:07:59.394614160-0400 127.0.0.1 80 127.0.0.1 42334 .....A.... 1 84
17 2026-09-08T10:07:59.451113955-0400 127.0.0.1 42334 127.0.0.1 80 .....A.... 84 389
18 2026-09-08T10:07:59.451391144-0400 127.0.0.1 42334 127.0.0.1 80 .....A...F 84 389
19 2026-09-08T10:07:59.451472334-0400 127.0.0.1 80 127.0.0.1 42334 .....A...F 389 85
20 2026-09-08T10:07:59.451499002-0400 127.0.0.1 42334 127.0.0.1 80 .....A.... 85 390
```

## Part D - Examine the HTTP Request and Response

5. I Filtererd for http.request and identify the GET method, request URI, Host and User-Agent.
6. I Filtered for http.response and record the status code, response phrase, server and content type.
7. And use Follow TCP Stream to reconstruct the complete request and response.
8. I also compare the TCP payload length with the sequence/acknowledgement progression.

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
4 2026-09-08T10:06:59.034714628-0400 127.0.0.1 56828 127.0.0.1 80 GET /basic.html 127.0.0.1 curl/8.21.0
14 2026-09-08T10:07:59.394505982-0400 127.0.0.1 42334 127.0.0.1 80 GET /basic.html 127.0.0.1 curl/8.21.0
```

```
tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
```

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
Running as user "root" and group "root". This could be dangerous.
6 2026-09-08T10:06:59.191703355-0400 200 OK Apache/2.4.68 (Debian) text/html 137
16 2026-09-08T10:07:59.451070329-0400 200 OK Apache/2.4.68 (Debian) text/html 137
```

```
tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
```

## Part E - Inspect Encapsulation and Connection Closure

9. Select the HTTP GET frame and expand Frame, Internet Protocol, Transmission Control Protocol and Hypertext Transfer Protocol.
10. Explain the relationship among frame length, IP total length, TCP header length and TCP payload length.
11. Locate FIN/ACK packets or a reset that closes the session. Explain the observed termination sequence.
12. State why a loopback capture does not provide normal source/destination MAC addresses. For MAC evidence, repeat with two authorized VMs on the same host-only network.

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
Running as user "root" and group "root". This could be dangerous.
8      2026-09-08T10:06:59.192132202-0400    127.0.0.1      56828    127.0.0.1      80      0x0011    84      389
9      2026-09-08T10:06:59.192486938-0400    127.0.0.1      80       127.0.0.1      56828   0x0011   389     85
18     2026-09-08T10:07:59.451391144-0400    127.0.0.1      42334    127.0.0.1      80      0x0011    84      389
19     2026-09-08T10:07:59.451472334-0400    127.0.0.1      80       127.0.0.1      42334   0x0011   389     85
```

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
```

**Explanation :** Each layer nests inside the one above it, frame length covers the entire Ethernet frame; IP total length is everything from the IP header onward; TCP header length is just the TCP header; TCP payload length is the actual HTTP data being carried. Each layer's header adds a small, fixed overhead on top of the layer it wraps.

**Why loopback does not provide real MAC addresses?** lo traffic never touches a physical or virtual network card at all, it's handled entirely inside the kernel, so there's no real Ethernet frame with genuine MAC addresses to observe.